

Integer Programming Study Set

1/9

Question 1. *There are six towns (towns 1–6) in Kings County. The county must determine where to build fire stations. The county wants to build the minimum number of fire stations needed to ensure that at least one fire station is within 15 minutes (driving time) of each town. The times (in minutes) required to drive between the towns in Kings County are shown in Table 1. Formulate an IP that will tell Kings how many fire stations should be built and where they should be located.*

From/To	town 1	town 2	town 3	town 4	town 5	town 6
town 1	0	10	20	30	30	20
town 2	10	0	25	35	20	10
town 3	20	25	0	15	30	20
town 4	30	35	15	0	15	25
town 5	30	20	30	15	0	14
town 6	20	10	20	25	14	0

Table 1: Time Required to Travel between towns in Kilroy County

Answer 1. *For each town, Kings County must determine whether to build a fire station there. We define the 0–1 variables x_1, x_2, x_3, x_4, x_5 , and x_6 by:*

$$x_i = \begin{cases} 1 & \text{if a fire station is built in town } i \\ 0 & \text{otherwise} \end{cases}$$

Then the total number of fire stations that are built is given by $x_1 + x_2 + x_3 + x_4 + x_5 + x_6$, and Kings's objective function is to:

$$\text{Minimize } Z = x_1 + x_2 + x_3 + x_4 + x_5 + x_6$$

Kings must ensure that there is a fire station within 15 minutes of each town. Table 1 indicates which locations can reach the town in 15 minutes or less. To ensure that at least one fire station is within 15 minutes of town 1, we add the constraint:

$$x_1 + x_2 \geq 1$$

The complete IP becomes:

$$\begin{array}{llll}
 \min & z = & x_1 + x_2 + x_3 + x_4 + x_5 + x_6 & \\
 \text{s.t.} & & x_1 + x_2 \geq 1 & \text{(town 1 constraint)} \\
 & & x_1 + x_2 + x_6 \geq 1 & \text{(town 2 constraint)} \\
 & & x_3 + x_4 \geq 1 & \text{(town 3 constraint)} \\
 & & x_3 + x_4 + x_5 \geq 1 & \text{(town 4 constraint)} \\
 & & x_4 + x_5 + x_6 \geq 1 & \text{(town 5 constraint)} \\
 & & x_2 + x_5 + x_6 \geq 1 & \text{(town 6 constraint)} \\
 & & x_i = 0 \text{ or } 1 (i = 1, 2, 3, 4, 5, 6) &
 \end{array}$$

Integer Programming Study Set

2/9

```
#Question 1

# Create a new model
m = Model("fire_station")

# Add variables
x = m.addVars(6, vtype=GRB.BINARY, name="x")

# Set the objective
m.setObjective(sum(x[i] for i in range(6)), GRB.MINIMIZE)

# Add constraints
m.addConstr(x[0] + x[1] >= 1, "town1Constraint")
m.addConstr(x[0] + x[1] + x[5] >= 1, "town2Constraint")
m.addConstr(x[2] + x[3] >= 1, "town3Constraint")
m.addConstr(x[2] + x[3] + x[4] >= 1, "town4Constraint")
m.addConstr(x[3] + x[4] + x[5] >= 1, "town5Constraint")
m.addConstr(x[1] + x[4] + x[5] >= 1, "town6Constraint")

# Solve model
m.optimize()

# Print solution
if m.status == GRB.Status.OPTIMAL:
    print('Optimal solution found:')
    for v in m.getVars():
        print(f'{v.varName}: {v.x}')
else:
    print('No optimal solution found')
```

Integer Programming Study Set

3/9

Question 2. Lower East Side Bakery plans to offer four sizes of cupcakes—tiny, small, midi, and large—each with its own profit margin and resource requirements detailed in Table 2. With 65 kg of dough, 25 kg of icing, and 800 labor hours available, the bakery must decide the quantity of each size to produce to maximize profits. The production is only viable economically if at least 500 units of a size are made. The goal is to construct an Integer Programming model to optimize the bakery's profits under these conditions.

	Cupcake Size			
	Tiny	Small	Midi	Large
Dough (in grams)	90	125	175	250
Icing (in grams)	35	50	75	90
Labor (in minutes)	1	1	1	1
Profit (\$)	1	2	3	4

Table 2: Resources and Profits for the Four Sizes of Cupcakes

Answer 2.

Let:

- x_1 be the number of tiny cupcakes prepared
- x_2 be the number of small cupcakes prepared
- x_3 be the number of midi cupcakes prepared
- x_4 be the number of large cupcakes prepared
- y_i is a binary variable is equal to 1 if we produce size i ; 0 otherwise (enforces the “viable only if at least 500” rule)

Lower East Side Bakery's objective is to maximize their profit with the cupcake sales. We know that if any cupcakes of a given size are produced, then at least 500 cupcakes of that size must be produced. Thus, for $i = 1, 2, 3, 4$ we must have $x_i \leq 0$ or $x_i \geq 500$. Then the IP becomes:

$$\begin{aligned}
 \max \quad & z = 2x_1 + 3x_2 + 4x_3 \\
 \text{s.t.} \quad & x_1 \leq My_1 \\
 & 500 - x_1 \leq M(1 - y_1) \\
 & x_2 \leq My_2 \\
 & 500 - x_2 \leq M(1 - y_2) \\
 & x_3 \leq My_3 \\
 & 500 - x_3 \leq M(1 - y_3) \\
 & x_4 \leq My_4 \\
 & 500 - x_4 \leq M(1 - y_4) \\
 & 90x_1 + 125x_2 + 175x_3 + 250x_4 \leq 65,000 \quad (\text{Dough constraint}) \\
 & 35x_1 + 50x_2 + 75x_3 + 90x_4 \leq 25,000 \quad (\text{Icing constraint}) \\
 & x_1 + x_2 + x_3 + x_4 \leq 800 \quad (\text{Labor constraint}) \\
 & x_1, x_2, x_3, x_4 \geq 0; \quad x_1, x_2, x_3, x_4 \text{ integer} \\
 & y_1, y_2, y_3, y_4 = 0 \text{ or } 1
 \end{aligned}$$

Integer Programming Study Set

4/9

```
from gurobipy import Model, GRB

# Create a new model
m = Model("bakery_model")

# Add variables
x1 = m.addVar(vtype=GRB.INTEGER, name="x1")
x2 = m.addVar(vtype=GRB.INTEGER, name="x2")
x3 = m.addVar(vtype=GRB.INTEGER, name="x3")
x4 = m.addVar(vtype=GRB.INTEGER, name="x4")
y1 = m.addVar(vtype=GRB.BINARY, name="y1")
y2 = m.addVar(vtype=GRB.BINARY, name="y2")
y3 = m.addVar(vtype=GRB.BINARY, name="y3")
y4 = m.addVar(vtype=GRB.BINARY, name="y4")

# Big M
M = 10000 # You might want to adjust this based on the problem context

# Set the objective
m.setObjective(2*x1 + 3*x2 + 4*x3, GRB.MAXIMIZE)

# Add constraints
m.addConstr(x1 <= M*y1, "c1")
m.addConstr(500 - x1 <= M*(1 - y1), "c2")
m.addConstr(x2 <= M*y2, "c3")
m.addConstr(500 - x2 <= M*(1 - y2), "c4")
m.addConstr(x3 <= M*y3, "c5")
m.addConstr(500 - x3 <= M*(1 - y3), "c6")
m.addConstr(x4 <= M*y4, "c7")
m.addConstr(500 - x4 <= M*(1 - y4), "c8")
m.addConstr(90*x1 + 125*x2 + 175*x3 + 250*x4 <= 65000, "DoughConstraint")
m.addConstr(35*x1 + 50*x2 + 75*x3 + 90*x4 <= 25000, "IcingConstraint")
m.addConstr(x1 + x2 + x3 + x4 <= 800, "LaborConstraint")

# Solve model
m.optimize()

# Print solution
if m.status == GRB.Status.OPTIMAL:
    print('Optimal solution found:')
    for v in m.getVars():
        print(f'{v.varName}: {v.x}')
else:
    print('No optimal solution found')
```

Integer Programming Study Set

5/9

Question 3. Suppose making a decision incurs a fixed cost (setup cost). For example, if a machine j produces x_j units, then the cost of using the machine as a function of x_j is:

$$f_j(x_j) = \begin{cases} 0 & \text{if } x_j = 0 \\ k_j + c_j x_j & \text{if } x_j > 0, \end{cases}$$

where k_j represents the fixed cost of using machine j and c_j represents the variable cost for machine j . The objective is to minimize total cost

$$\sum_{j=1}^N f_j(x_j).$$

How do we model this?

Answer 3. If it weren't for the setup cost k_j , this problem would be a simple LP. We can use binary variables to help the situation:

$$y_j = \begin{cases} 1 & \text{if } x_j > 0 \\ 0 & \text{if } x_j = 0. \end{cases}$$

Now the objective function is

$$\sum_{j=1}^N (k_j y_j + c_j x_j).$$

We've taken care of the objective function, but how to do we make sure that the x and y values work with each other? After all, we can't have a situation where $y_j = 0$ and $x_j = 10$. If $y_j = 0$, then x_j must be 0, too. If $y_j = 1$, then x_j can take on any value other than zero:

$$x_j \leq M y_j \quad \forall j \in \{1, \dots, N\}$$

$$x_j \geq \epsilon_j y_j \quad \forall j \in \{1, \dots, N\}$$

$$y_j \in \{0, 1\} \quad \forall j \in \{1, \dots, N\}.$$

Here M stands for a very large number and ϵ stands for a very small number.

Integer Programming Study Set

6/9

Question 4. *The Lotus Point Condo Project will contain both homes and apartments. The site can accommodate up to 10,000 dwelling units. The project must contain a recreation project: either a swimming-tennis complex or a sailboat marina, but not both. If a marina is built, then the number of homes in the project must be at least triple the number of apartments in the project. A marina will cost \$1.2 million, and a swimming-tennis complex will cost \$2.8 million. The developers believe that each apartment will yield revenues with an NPV of \$48,000, and each home will yield revenues with an NPV of \$46,000. Each home (or apartment) costs \$40,000 to build. Formulate an IP to help Lotus Point maximize profits.*

Answer 4. *Let $x = 1$ denote the case where a marina will be built and $x = 0$ denote the case where a swimming-tennis complex will be built. Let v be number of apartments and u be number of houses.*

In units of \$1,000, the total profit would be

$$48v + 46u - 40(u + v) - 1200x - 2800(1 - x)$$

The constraint must satisfy $u \geq 3v$ if $x = 1$. This "if" constraint can be formulated as

$$u \geq 3v - M(1 - x).$$

Lastly, the total number of apartments and houses must satisfy $u + v \leq 10000$

The whole formulation can be written as

$$\begin{aligned} &\text{maximize } 48v + 46u - 40(u + v) - 1200x - 2800(1 - x) \\ &\text{subject to } u + v \leq 10000 \\ &\quad u \geq 3v - M(1 - x) \\ &\quad x \in \{0, 1\}, u, v \in \mathbb{Z}_+ \end{aligned}$$

Integer Programming Study Set

7/9

```
from gurobipy import Model, GRB

# Create a new model
model = Model("lotus")

# Add variables
u = model.addVar(vtype=GRB.INTEGER, name="u")
v = model.addVar(vtype=GRB.INTEGER, name="v")
x = model.addVar(vtype=GRB.BINARY, name="x")

# Set objective
model.setObjective(48*v + 46*u - 40*(u + v) - 1200*x - 2800*(1 - x), GRB.
    MAXIMIZE)

# Add constraints
model.addConstr(u + v <= 10000, "constraint1")

# Add a large constant M for the big-M method
M = 100000 # This value might need to be adjusted depending on the context
model.addConstr(u >= 3*v - M*(1 - x), "constraint2")

# Optimize model
model.optimize()

# Print solution
if model.status == GRB.Status.OPTIMAL:
    print('Optimal solution found:')
    for var in model.getVars():
        print(f'{var.varName}: {var.x}')
    print(f'Objective: {model.objVal}')
else:
    print('No optimal solution found')
```

Integer Programming Study Set

8/9

Question 5. Columbia IEOR has a \$20,000 advertising budget. Columbia can purchase full-page ads in two magazines: Indian Journal (IJ) and Fenghua Shibao (FS). An exposure occurs when a person reads a Columbia ad for the first time. The number of exposures generated by each ad in IJ is as follows:

- ads 1–6, 10,000 exposures;
- ads 7–10, 3,000 exposures;
- ads 11–15, 2,500 exposures;
- ads 16+, 0 exposures.

For example, 8 ads in IJ would generate $6(10,000) + 2(3,000) = 66,000$ exposures. The number of exposures generated by each ad in FS is as follows:

- ads 1–4, 8,000 exposures;
- ads 5–12, 6,000 exposures;
- ads 13–15, 2,000 exposures;
- ads 16+, 0 exposures.

Thus, 13 ads in FS would generate $4(8,000) + 8(6,000) + 1(2,000) = 82,000$ exposures. Each full-page ad in either magazine costs \$1,000. Assume there is no overlap in the readership of the two magazines. Formulate an IP to maximize the number of exposures that Columbia IEOR can obtain with limited advertising funds.

Answer 5.

Magazines and tier caps.

<i>IJ caps</i>	$c_1 = 6$	$c_2 = 4$	$c_3 = 5$
<i>FS caps</i>	$f_1 = 4$	$f_2 = 8$	$f_3 = 3$

Per-ad exposures by tier.

$$(IJ) (\alpha_1, \alpha_2, \alpha_3) = (10,000, 3,000, 2,500), \quad (FS) (\beta_1, \beta_2, \beta_3) = (8,000, 6,000, 2,000).$$

Decision variables.

$$\begin{aligned} s_t^{IJ}, s_t^{FS} &\in \mathbb{Z}_{\geq 0} \quad (t = 1, 2, 3), \\ z_t^{IJ}, z_t^{FS} &\in \{0, 1\} \quad (t = 1, 2, 3). \end{aligned}$$

Model.

$$\begin{aligned} \max \quad & \alpha_1 s_1^{IJ} + \alpha_2 s_2^{IJ} + \alpha_3 s_3^{IJ} + \beta_1 s_1^{FS} + \beta_2 s_2^{FS} + \beta_3 s_3^{FS} \\ \text{s.t.} \quad & s_1^{IJ} + s_2^{IJ} + s_3^{IJ} + s_1^{FS} + s_2^{FS} + s_3^{FS} \leq \text{total ads}, \quad (\text{budget}) \\ & 0 \leq s_1^{IJ} \leq c_1 z_1^{IJ}, \quad 0 \leq s_2^{IJ} \leq c_2 z_2^{IJ}, \quad 0 \leq s_3^{IJ} \leq c_3 z_3^{IJ}, \\ & 0 \leq s_1^{FS} \leq f_1 z_1^{FS}, \quad 0 \leq s_2^{FS} \leq f_2 z_2^{FS}, \quad 0 \leq s_3^{FS} \leq f_3 z_3^{FS}, \quad (\text{tiers open ads}) \\ & z_2^{IJ} \leq z_1^{IJ}, \quad z_3^{IJ} \leq z_2^{IJ}, \quad z_2^{FS} \leq z_1^{FS}, \quad z_3^{FS} \leq z_2^{FS}, \quad (\text{tier order}) \\ & s_1^{IJ} \geq c_1 z_2^{IJ}, \quad s_2^{IJ} \geq c_2 z_3^{IJ}, \\ & s_1^{FS} \geq f_1 z_2^{FS}, \quad s_2^{FS} \geq f_2 z_3^{FS}, \quad (\text{fill lower caps before higher}) \\ & z_t^{IJ}, z_t^{FS} \in \{0, 1\}, \quad s_t^{IJ}, s_t^{FS} \in \mathbb{Z}_{\geq 0}. \end{aligned}$$

Integer Programming Study Set

9/9

Question 6. Recall the binary Knapsack problem we have covered in class. Suppose we have unlimited units of the three items shown in Table 3. The weight capacity of the knapsack is 5. Using IP, determine how many units of each item should go into the knapsack to maximize the total benefit.

Item (n)	Weight (w_n)	Benefit (b_n)
1	2	65
2	3	80
3	1	30

Table 3: Items for the knapsack problem.

Answer 6. Let x_1, x_2, x_3 be the number of units of items 1, 2, and 3, respectively, that are included in the knapsack. These are integer variables since we are dealing with discrete items.

We aim to maximize the total benefit. The benefit of each item is given, so the objective function is: Maximize $Z = 65x_1 + 80x_2 + 30x_3$

$$\begin{aligned}
 &\text{Maximize} && Z = 65x_1 + 80x_2 + 30x_3 \\
 &\text{subject to} && 2x_1 + 3x_2 + x_3 \leq 5, \\
 &&& x_1, x_2, x_3 \geq 0, \\
 &&& x_1, x_2, x_3 \text{ are integers.}
 \end{aligned}$$

```

from gurobipy import Model, GRB

# Create a new model
m = Model("knapsack")

# Create variables
x1 = m.addVar(vtype=GRB.INTEGER, name="x1")
x2 = m.addVar(vtype=GRB.INTEGER, name="x2")
x3 = m.addVar(vtype=GRB.INTEGER, name="x3")

# Set objective
m.setObjective(65 * x1 + 80 * x2 + 30 * x3, GRB.MAXIMIZE)

# Add constraint: 2x1 + 3x2 + x3 <= 5
m.addConstr(2 * x1 + 3 * x2 + x3 <= 5, "c0")

# Optimize model
m.optimize()

# Print solution
if m.status == GRB.OPTIMAL:
    print(f"Optimal solution: x1={x1.X}, x2={x2.X}, x3={x3.X}")
    print(f"Maximum benefit: {m.ObjVal}")
else:
    print("No optimal solution found.")

```